

Structure optimale pour un mode deux joueurs avec réactions aux attaques : Principes et architecture

1. Concepts fondamentaux du mode multijoueur

Architecture Client-Serveur

La première considération pour un mode deux joueurs est de maintenir une architecture client-serveur solide où **toute la logique de jeu reste sur le serveur**. Dans le cas présent, le serveur NestJS existant est idéal car il centralise déjà les règles et l'état du jeu.

État partagé et synchronisation

Le jeu doit maintenir un état unique, autoritaire, qui est:

- Stocké sur le serveur
- Synchronisé avec tous les clients via des événements WebSocket
- Protégé contre les manipulations côté client

2. Modèle de données pour le multijoueur

Extension du modèle GameState

Le modèle de données actuel doit être enrichi pour supporter:

- Une collection de plusieurs joueurs plutôt qu'un seul joueur actif
- Un système d'indexation pour suivre le joueur actif
- Des états supplémentaires liés aux interactions entre joueurs

Gestion des joueurs

Chaque joueur doit avoir:

- Un identifiant unique
- Un statut indiquant s'il est actif ou en attente
- Des capacités spécifiques selon la phase du jeu (ex: possibilité de réagir)

3. Système de tours alternatifs

Gestion des tours

Un système de tours doit être implémenté pour:

- Alternier l'accès aux actions entre les joueurs
- Réinitialiser les états spécifiques au tour (comme "a déjà pioché")
- Transmettre clairement l'information sur le joueur actif

Fin de tour contrôlée

La fin d'un tour doit:

- Être explicitement initiée par le joueur ou déclenchée automatiquement
- Effectuer les vérifications nécessaires avant de passer au joueur suivant
- Déclencher des événements spécifiques (ex: application d'effets persistants)

4. Mécanisme d'attaque et de réaction

Système d'attaque

Un joueur actif doit pouvoir:

- Cibler spécifiquement l'adversaire avec certaines cartes
- Créer un "état d'attaque en attente" dans le système
- Voir son tour temporairement suspendu pendant l'attente d'une réaction

Système de réaction

Le joueur ciblé doit avoir:

- Un délai limité pour réagir (minuteur côté serveur)
- Des options de réaction basées sur ses cartes disponibles
- La possibilité d'accepter, bloquer ou contre-attaquer

Timing de réaction

Le système de réaction doit:

- Avoir un minuteur précis géré par le serveur
- Appliquer une action par défaut si le délai expire
- Revenir au joueur actif après résolution

5. Communication et flux de données

Événements WebSocket

Le système doit être étendu avec de nouveaux événements:

- `initiateAttack`
: Déclenché quand un joueur attaque
- `reaction`
: Envoyé par un joueur qui réagit
- `switchTurn`
: Signal pour passer au joueur suivant

Sécurité et validation

Chaque action doit être:

- Validée côté serveur pour confirmer sa légitimité
- Vérifiée dans le contexte du joueur (ex: est-ce vraiment son tour?)
- Protégée contre les manipulations (temps de réaction, cartes utilisées)

6. Interface utilisateur réactive

Statut du joueur

L'interface doit clairement indiquer:

- Quel joueur est actif
- Si une réaction est attendue et de qui
- Le temps restant pour réagir

Visualisation de l'adversaire

L'interface doit montrer:

- Les informations essentielles de l'adversaire (santé, cartes en main)
- Les actions récentes effectuées par l'adversaire
- Un état visuel différent quand c'est le tour de l'adversaire

Contexte d'attaque et de défense

Lors d'une attaque, l'interface doit:

- Mettre en évidence la relation attaquant-défenseur
- Présenter clairement les options de réaction disponibles
- Indiquer visuellement le temps restant pour réagir

7. Système de notification

Messages contextuels

Le jeu doit fournir:

- Des messages clairs sur les actions en cours
- Des indications sur qui doit agir maintenant
- Des informations sur ce qui vient de se produire

Feedback sonore

Des indications sonores distinctes pour:

- L'initiation d'une attaque
- La réception d'une attaque nécessitant une réaction
- Le succès ou l'échec d'une défense

8. Gestion des états spéciaux

Interruption temporaire du flux de jeu

Le système doit:

- Suspendre le tour normal pendant l'attente d'une réaction
- Reprendre correctement après résolution
- Gérer les cas particuliers (ex: joueur déconnecté pendant une attente)

Résolution des interactions

La résolution des interactions doit:

- Appliquer les effets dans un ordre déterministe
- Calculer correctement les résultats des contre-attaques
- Mettre à jour l'état global après résolution

Conclusion

L'intégration d'un mode deux joueurs avec système de réaction ne nécessite pas une refonte complète, mais plutôt une extension de l'architecture existante. En gardant la logique de jeu centralisée sur le serveur et en ajoutant des couches supplémentaires pour gérer les interactions entre joueurs, le système peut évoluer tout en conservant sa robustesse.

Le défi principal est de créer un flux de jeu alternant fluide, avec des interruptions contrôlées pour les réactions, tout en maintenant une expérience utilisateur claire et informative. Cela passe par une communication efficace entre serveur et clients, des validations strictes côté serveur, et une interface qui reflète fidèlement l'état actuel du jeu et les actions disponibles pour chaque joueur.